Plano de Implantação - PortfolioHUB

1. **Objetivo.** Implantar o PortfolioHUB como plataforma centralizada para exibir e gerenciar meus projetos e portfólios digitais, com gestão de usuários, segurança e controle de acesso via Git e GitHub.
2. **Escopo e componentes**
    - Repositório GitHub *portfolio-academico* (armazenamento de projetos)
    - Site no GitHub Pages (vitrine pública)
    - Google Docs (plano de documentação), Google Drive (evidências) e YouTube (apresentação)
    - Google Gemini como guia de implantação
3. **Etapas, prazos e entregáveis**

Etapa	Prazo	Entregável
1. Planejamento	22/10	Plano + Gemini configurado
2. Configuração e GitHub	22/10	Estrutura do repositório e README
3. Usuários e segurança	23/10	2FA, papéis, proteção da *main*, *SECURITY.md*
4. Compartilhamento e acesso	23/10	Pull request real, *CONTRIBUTING.md*
5. Finalização e testes	24/10	Checklist de testes, versão 2.0 publicada
6. Apresentação e PDF	25/10	Vídeo no YouTube e PDF entregue

4. **Riscos e mitigação**

- Perder acesso à conta: ativar 2FA e guardar os códigos de recuperação.

- Vazamento de dados sensíveis: .gitignore e secret scanning.
- Alteração indevida na main: proteção de branch e revisão por pull request.
- Atraso no cronograma: terminar o PDF na sexta e deixar sábado e domingo para o vídeo.

5. **Métricas de sucesso:** 100% dos itens do checklist de testes aprovados; main protegida; todos os projetos com README; site publicado e acessível; vídeo de até 5 minutos publicado.
6. **Pós-implantação:** atualizar o portfólio a cada novo projeto, revisar permissões periodicamente e responder issues.
7. **Revisão do Gemini.**

Etapa 1 – Planejamento (Prompt 1). O Gemini foi configurado como guia, com o contexto da disciplina, do desafio e das entregas anteriores, e revisou o plano de implantação, avaliando-o como bem estruturado. Apontou quatro lacunas: (1) configuração do Git local e da autenticação com o GitHub, já atendida, pois o Git está instalado e configurado e os envios ao repositório funcionam; (2) roteiro para o vídeo de apresentação, incluído na Etapa 6; (3) teste de responsividade do site, incluído no checklist de testes da Etapa 5.

Etapa 2 – Configuração e GitHub (Prompt 2). O Gemini avaliou a estrutura do repositório como limpa e lógica e sugeriu: (1) READMEs individuais por projeto; (2) centralizar na pasta `documentacao` os artefatos do Google Workspace; (3) mensagens de commit descritivas. As três foram aplicadas. A sugestão de incluir GIFs e capturas de tela nos READMEs não foi aplicada por ora, pois não há projetos com interface visual.

Etapa 3 – Usuários e segurança (Prompt 3). O Gemini propôs uma política com sete itens: autenticação em duas etapas, controle de acesso, proteção da

branch `main`, prevenção de vazamentos (`.gitignore` e secret scanning), Dependabot, LGPD e `SECURITY.md`. Todos foram implementados. Duas orientações foram verificadas e ajustadas: (a) o Gemini sugeriu o papel "Triage" para o colaborador, mas repositórios de contas pessoais oferecem apenas o nível de colaborador com permissão de escrita, e papéis granulares exigem uma organização; por isso o controle de acesso se apoia na proteção da `main`; (b) o secret scanning, citado como ativo por padrão, foi conferido nas configurações do repositório e está ativo, junto com o push protection. O Dependabot foi ativado, mas não há arquivos de dependências, portanto não gera alertas no momento.

Etapa 4 – Compartilhamento e controle de acesso (Prompt 4). O Gemini avaliou o fluxo (issue, branch, pull request e revisão) como compatível com a prática de mercado e sugeriu: template de pull request, CODEOWNERS, labels e convenção de commits. Foram aplicados o template de PR, a convenção de commits (no `CONTRIBUTING.md`) e a label [`documentation`, se aplicar]. O CODEOWNERS não foi adotado, pois o repositório tem uma única proprietária e a exigência de revisão por code owners bloquearia as próprias alterações dela. A orientação de usar "Require review from Code Owners" para impedir autoaprovação também foi corrigida: o GitHub já impede que o autor aprove o próprio pull request.

Problema encontrado e correção. O primeiro pull request (`CONTRIBUTING.md`), aberto pela conta colaboradora, foi mergeado pela proprietária sem revisão. Ao investigar, constatou-se que o ruleset da `main` estava com status "Disabled" e sem branch alvo, portanto não estava em vigor. [Descreva o que você corrigiu de fato.] O fluxo foi repetido com o template de pull request (PR #10): o merge ficou bloqueado até a aprovação da conta colaboradora.